TUTORIAL 01

Java Recap

Object-oriented programming, exceptions, and the Streams API — the core Java you need before writing Spring Boot services.

OOP · Encapsulation · Inheritance · Polymorphism · Abstraction · Exceptions · Streams

Table of Contents

1. Objects & Classes

Everything in Java lives inside a **class**. A class is a blueprint; an **object** is a concrete thing built from that blueprint. The blueprint describes two things: **state** (fields) and **behaviour** (methods).

```
public class Seat {

    // ---- state (fields) ----
    private String label;      // e.g. "A12"
    private boolean booked;

    // ---- constructor: runs when you write "new Seat(...)" ----
    public Seat(String label) {
        this.label = label;    // "this" = the object being built
        this.booked = false;
    }

    // ---- behaviour (methods) ----
    public void book() {
        this.booked = true;
    }

    public boolean isBooked() {
        return booked;
    }
}
```

```
// Creating and using objects
Seat a12 = new Seat("A12");   // "a12" is a reference to a Seat object
a12.book();                   // call a method on it
System.out.println(a12.isBooked());  // -> true
```

The four things every class can hold

Member	What it is	Example
Field	A variable that stores object state	`private String label;`
Constructor	Special method that initialises a new object	`public Seat(String label)`
Method	A named block of behaviour	`public void book()`
Static member	Belongs to the class, not an instance	`public static int count;`

> **NOTE**
>
> `static` members are shared by *all* objects of the class and are accessed through the class name (`Math.max(...)`), not through an instance. Instance members belong to one object.

The four pillars of OOP

The rest of this tutorial walks through the four principles you will hear about in every interview. Keep this mental model:

Pillar	One-line idea
Encapsulation	Hide the data, expose a safe API.
Inheritance	Build a new class on top of an existing one.
Polymorphism	One name, many behaviours — chosen at runtime.
Abstraction	Program to a concept, not a concrete class.

2. Encapsulation

Encapsulation means keeping an object's fields **private** and only letting the outside world touch them through methods you control. This lets you enforce rules and change the internals later without breaking callers.

> **WATCH OUT**
> Public mutable fields are the classic mistake. If any code can write `seat.booked = true`, you can never guarantee an invariant (e.g. “a cancelled seat can’t be re-booked”). Make fields `private`.

```
public class Account {

    private long balanceCents;   // hidden state

    public void deposit(long cents) {
        if (cents <= 0) {
            throw new IllegalArgumentException("deposit must be positive");
        }
        balanceCents += cents;
    }

    public void withdraw(long cents) {
        if (cents > balanceCents) {
            throw new IllegalStateException("insufficient funds");
        }
        balanceCents -= cents;
    }

    // read-only access: a "getter" with no matching public setter
    public long getBalanceCents() {
        return balanceCents;
    }
}
```

The rules (“deposit must be positive”, “cannot overdraw”) live *inside* the object. No caller can put the account into an invalid state.

Getters, setters and records

A plain data holder often just needs getters and setters. Since Java 16, a `record` generates them — plus `equals`, `hashCode` and `toString` — for you, and its fields are immutable:

```java
// One line replaces ~40 lines of boilerplate.
public record Money(String currency, long amountCents) { }

Money price = new Money("USD", 4500);
price.currency();   // accessor is named after the field -> "USD"
```

TIP

Records are perfect for DTOs and value objects. Use a normal class with private fields when the object has real behaviour and mutable state.

3. Inheritance

Inheritance lets a class reuse and extend another class with the `extends` keyword. The child (*subclass*) gets all non-private members of the parent (*superclass*) and can add or override behaviour.

```
public class Ticket {
    protected final String holder;

    public Ticket(String holder) {
        this.holder = holder;
    }

    public String describe() {
        return "Ticket for " + holder;
    }
}

public class VipTicket extends Ticket {
    private final String loungeAccess;

    public VipTicket(String holder, String lounge) {
        super(holder);            // call the parent constructor first
        this.loungeAccess = lounge;
    }

    @Override
    public String describe() {     // override the parent method
        return super.describe() + " (VIP, lounge " + loungeAccess + ")";
    }
}
```

Keyword	Meaning
`extends`	Declares a subclass of one superclass.
`super(...)`	Calls the superclass constructor (must be the first statement).
`super.method()`	Calls the superclass version of an overridden method.
`@Override`	Compiler check that you really are overriding something.
`final` (class)	Prevents any subclassing.
`protected`	Visible to subclasses and the same package.

NOTE

Java has **single inheritance** for classes — a class can extend only one class — but a class can implement *many* interfaces (Section 5). Every class implicitly extends `java.lang.Object`.

TIP

Favour composition over inheritance. If “B *is-a* A” is not literally true, hold an A as a field instead of extending it. Inheritance couples the subclass tightly to the parent’s internals.

4. Polymorphism

Polymorphism — “many forms” — means the same call can run different code depending on the actual object. Java has two flavours.

Compile-time: method overloading

Same method name, different parameter lists. The compiler picks the match.

```
int    max(int a, int b)          { return a > b ? a : b; }
double max(double a, double b)    { return a > b ? a : b; }
String max(String a, String b)    { return a.compareTo(b) >= 0 ? a : b; }
```

Run-time: method overriding & dynamic dispatch

This is the important one. A variable of a parent type can hold any subclass object, and the *object's* method runs — decided at runtime, not compile time.

```
Ticket t = new VipTicket("Sara", "West");   // parent-typed reference
System.out.println(t.describe());
// -> "Ticket for Sara (VIP, lounge West)"   // VipTicket.describe() runs
```

```
List<Ticket> tickets = List.of(
    new Ticket("Ali"),
    new VipTicket("Sara", "West")
);

// One loop, correct behaviour per object -- no if/else on type.
for (Ticket ticket : tickets) {
    System.out.println(ticket.describe());
}
```

> **TIP**
>
> Dynamic dispatch is the engine behind clean design: write code against a general type (`Ticket` , `PaymentGateway` , `List`) and let each concrete implementation supply the behaviour. This is exactly how Spring lets you swap a real service for a fake one in tests.

	Overloading	Overriding
Resolved	At compile time	At run time
Signatures	Must differ	Must match exactly
Relationship	Same class	Parent → child

5. Abstraction

Abstraction means exposing *what* an object does while hiding *how*. In Java you express it with **abstract classes** and **interfaces**.

Interfaces — a pure contract

```
public interface PaymentGateway {
    PaymentResult charge(long amountCents, String token);
}

// Two interchangeable implementations of the same contract:
public class StripeGateway implements PaymentGateway {
    public PaymentResult charge(long amountCents, String token) { /* real HTTP call */ }
}

public class FakePaymentGateway implements PaymentGateway {
    public PaymentResult charge(long amountCents, String token) {
        return PaymentResult.approved();   // used in tests
    }
}
```

Code that depends only on `PaymentGateway` neither knows nor cares which implementation it gets — that decision is made elsewhere (in Spring, by the container). This is **abstraction** and **polymorphism** working together.

Abstract classes — partial implementation

```
public abstract class Shape {
    public abstract double area();          // no body: subclass MUST provide it

    public String summary() {               // shared, concrete behaviour
        return getClass().getSimpleName() + " area=" + area();
    }
}

public class Circle extends Shape {
    private final double r;
    public Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
}
```

	Interface	Abstract class
Instantiable?	No	No
Fields	Only constants	Any instance fields
State/constructor	No	Yes
How many?	Implement many	Extend only one
Method bodies	`default` methods only	Any concrete methods

TIP

Rule of thumb: reach for an *interface* to describe a capability (“can be charged”, “can be compared”). Reach for an *abstract class* when subclasses share real code and state — like the `BaseEntity` superclass every entity extends in the event-ticketing project.

6. Exceptions

An exception is Java's way of signalling “something went wrong”. It unwinds the call stack until some `catch` block handles it — or the program stops.

The hierarchy: checked vs unchecked

```
Throwable
 |-- Error                // JVM-level, do NOT catch (OutOfMemoryError...)
 |-- Exception            // CHECKED: must be declared or handled
 |     |-- IOException, SQLException, ...
 |     |-- RuntimeException // UNCHECKED: optional to handle
 |           |-- NullPointerException, IllegalArgumentException,
 |           |   IllegalStateException, ...
```

	Checked	Unchecked (RuntimeException)
Compiler forces handling?	Yes	No
Typical cause	External/recoverable (I/O, network)	Programming bug or broken invariant
Examples	`IOException`	`NullPointerException`, `IllegalArgumentException`

try / catch / finally

```
try {
    var result = gateway.charge(4500, token);   // may throw
    return result;
} catch (PaymentDeclinedException e) {
    log.warn("declined: {}", e.getMessage());    // handle specific first
    return PaymentResult.declined();
} catch (Exception e) {
    throw new IllegalStateException("payment failed", e);  // wrap + keep cause
} finally {
    meter.record();   // ALWAYS runs -- cleanup, closing, metrics
}
```

try-with-resources

Anything implementing `AutoCloseable` is closed automatically — no `finally` needed:

```java
try (var reader = Files.newBufferedReader(path)) {
    return reader.readLine();
}   // reader.close() is called for you, even on exception
```

Throwing and custom exceptions

```java
public class SeatUnavailableException extends RuntimeException {
    public SeatUnavailableException(String seatLabel) {
        super("Seat " + seatLabel + " is no longer available");
    }
}

public void hold(Seat seat) {
    if (seat.isBooked()) {
        throw new SeatUnavailableException(seat.getLabel());
    }
    // ...
}
```

TIP

Prefer **unchecked** exceptions for business-rule violations in Spring apps. The event-ticketing project does exactly this: `ResourceNotFoundException` and `ConflictException` extend `RuntimeException`, and a single `@RestControllerAdvice` turns them into clean HTTP responses (see Tutorial 02).

WATCH OUT

Never `catch (Exception e) {{ }}` and swallow it silently. At minimum log it, or rethrow. A swallowed exception is a bug you will never find.

7. The Streams API

A **stream** is a pipeline for processing a sequence of elements declaratively — you describe *what* you want, not the loop mechanics. Streams came in Java 8 and are everywhere in modern code.

The shape of every pipeline

```
source            .intermediate ops.          terminal op
list.stream()     .filter(...).map(...)       .toList();
```

- **Source** — `collection.stream()`, `Stream.of(...)`, `Arrays.stream(...)`.
- **Intermediate** — lazy, return a new stream: `filter`, `map`, `sorted`, `distinct`, `limit`.
- **Terminal** — triggers execution and produces a result: `collect`, `toList`, `count`, `reduce`, `forEach`.

The workhorses: filter, map, collect

```
List<Ticket> tickets = ...;

// Names of VIP holders, sorted, as a List
List<String> vipNames = tickets.stream()
        .filter(t -> t instanceof VipTicket)   // keep matching elements
        .map(Ticket::holder)                   // transform each element
        .sorted()
        .toList();                             // terminal -> List<String>
```

This is the same pattern the event-ticketing services use, e.g. `repository.findAll().stream().map(OrganizerResponse::from).toList()` to turn entities into response DTOs.

Lambdas & method references

```
t -> t.holder()          // lambda: (params) -> body
Ticket::holder           // method reference: shorthand for the lambda above
System.out::println      // reference to an existing method
Money::new               // constructor reference
```

reduce & the numeric shortcuts

```
long totalCents = orders.stream()
        .mapToLong(Order::amountCents)   // -> LongStream
        .sum();                          // 0, count(), average(), max()...

// General reduce: (identity, accumulator)
int product = Stream.of(1, 2, 3, 4).reduce(1, (a, b) -> a * b);  // 24
```

Collectors — beyond a plain list

```
import static java.util.stream.Collectors.*;

Map<Boolean, List<Ticket>> byVip =
        tickets.stream().collect(partitioningBy(t -> t instanceof VipTicket));

Map<String, List<Ticket>> byHolder =
        tickets.stream().collect(groupingBy(Ticket::holder));

String csv = names.stream().collect(joining(", ", "[", "]"));
```

Optional — no more NullPointerException

`Optional<T>` is a box that holds a value *or nothing*. It forces the caller to deal with the “absent” case:

```
Optional<Ticket> found = tickets.stream()
        .filter(t -> t.holder().equals("Sara"))
        .findFirst();

// Handle both cases explicitly:
Ticket t = found.orElseThrow(() -> new ResourceNotFoundException("no ticket"));

found.map(Ticket::describe).ifPresent(System.out::println);
```

WATCH OUT

Streams are **single-use** and should be side-effect free. Don’t mutate external state inside `map` / `filter`, and don’t reuse a stream after a terminal operation — create a fresh one.

8. Cheat Sheet

ACCESS MODIFIERS

Modifier	Visible from
`public`	Everywhere
`protected`	Same package + subclasses
(none)	Same package only
`private`	Same class only

OOP IN ONE LINE EACH

Encapsulation	Private fields + public methods that guard invariants.
Inheritance	`class Child extends Parent`; reuse + `@Override`.
Polymorphism	Parent-typed reference runs the child's method (dynamic dispatch).
Abstraction	Depend on `interface` / `abstract`, not concretes.

EXCEPTIONS

- **Checked** = must handle/declare (I/O). **Unchecked** = `RuntimeException`, optional.
- `try / catch (most-specific first) / finally`; `try (resource)` auto-closes.
- Wrap with a cause: `new XException("msg", e)`. Never swallow silently.

STREAMS

- Pipeline = **source** → **intermediate** (lazy) → **terminal** (runs it).
- Core ops: `filter`, `map`, `sorted`, `reduce`, `collect`, `toList`.
- `Optional` models "maybe a value": `map`, `orElse`, `orElseThrow`, `ifPresent`.

> **TIP**
>
> **Next:** Tutorial 02 shows how Spring Boot uses these ideas — interfaces, dependency injection and streams — to wire a real REST application together.